

Nested Structures

Learning Objectives

- Work with nested data structures: lists of dictionaries, dictionaries of lists, and deeper nesting
- Navigate and manipulate complex nested data efficiently
- Design hierarchical data models for real-world applications

Engage (5-7 minutes)

Real-world data is rarely flat. A school has classes, each class has students, each student has subjects, and each subject has marks. This hierarchical structure requires nested data structures. Today we learn to work with deeply nested data, navigate complex structures, and build models that mirror real-world complexity.

Explore (10-12 minutes)

Create this nested structure: a dictionary representing a school with three classes, each class containing a list of students, each student being a dictionary with name, age, and marks (a dictionary of subject: score). Access the marks of the second student in the second class for Science. Now iterate through all students and calculate their averages. How many levels of nesting did you need to traverse?

Explain (10-12 minutes)

Nested structures combine multiple data types to represent complex relationships. A list of dictionaries is common for tabular data (each dictionary is a row). A dictionary of lists groups data by category. Deeper nesting (3+ levels) models hierarchical relationships but should be used carefully—excessive nesting makes code hard to read and maintain. Accessing nested data requires chaining operations: `data["class1"]["students"][0]["name"]`. Use `.get()` for safe access to avoid `KeyError` in deeply nested structures.

Elaborate (8-10 minutes)

Build a comprehensive school management system with nested structures: school → classes → students → subjects → grades. Implement functions to: add a new student to any class, update a student's grade, find the class with the highest average, list all students who scored above 90 in any subject, and generate a complete report card. Handle edge cases like missing data and empty classes. Then flatten the nested structure into a simple list for easy searching. Discuss: at what point does nesting become too complex?

Evaluate (5-8 minutes)

Given a nested structure representing a company with departments, teams, and employees, write code to: access an employee's name given department and team names, count total employees across all departments, find the department with the most employees, and create a flat list of all employee names. Explain the trade-offs of nested versus flat structures.